Project Report: Apartment Rent Price Prediction

Hasti Azadnia
Student Number: S336624
Politecnico di Torino
Data Science Lab: Process and methods
Email: s338984@studenti.polito.it

Ayda Ghasemazar
Student Number: S338736
Politecnico di Torino
Data Science Lab: Process and methods
Email: ghasemazar.ayda@studenti.polito.it

***Abstract*—This project involves developing a machine learning model to predict monthly apartment rental prices in the U.S. using a large dataset of nearly 100,000 listings. A detailed data preprocessing and feature engineering pipeline was established, converting numerical, categorical, and textual data into predictive features. A LightGBM regressor was trained on the log-transformed prices, with text features handled via TF–IDF vectorization and categorical variables processed using target encoding. The model was optimized through outlier management and iterative hyperparameter tuning, achieving a strong predictive performance, indicated by a Mean Absolute Error (MAE) of approximately $162.06.**

I. Introduction

Accurate rental price predictions are crucial for effective real estate decisions, impacting property management, investment strategies, and urban planning. This project develops a user-friendly machine learning model to forecast monthly apartment rents using location, size, amenities, and listing details. The model's accuracy is evaluated primarily with Mean Absolute Error (MAE).

A. Dataset Overview & EDA

The dataset consists of 99,487 apartment rent announcements from across the United States, divided into development.csv (79,589 rows, 21 columns) and evaluation.csv (19,898 rows, 20 columns, lacking the price target). Features include numerical (bathrooms, bedrooms, square_feet, latitude, longitude, time), categorical (category, currency, fee, has_photo, pets_allowed, price_type, cityname, state, source), and textual (title, body, amenities).

Initial Exploratory Data Analysis (EDA) revealed:

- **Missing Values**: address was 92% missing (unusable), pets_allowed 60-61% missing, and amenities 16% missing. Other columns had $< 1\%$ missingness.
- **Target Variable (price)**: Mean $\approx$ \$1525, Std $\approx$ \$892, Min = \$100, Max = \$52500. The large standard deviation and significant difference from the median ($\approx$ \$1350) indicated a right-skewed distribution and the presence of outliers.
- **Feature Distributions**: square_feet (Mean $\approx$ 955, Std $\approx$ 391, Max=40000) also showed potential outliers. category was extremely imbalanced, rendering it uninformative. currency and price_type had single unique values. cityname exhibited high cardinality (2806 unique values).

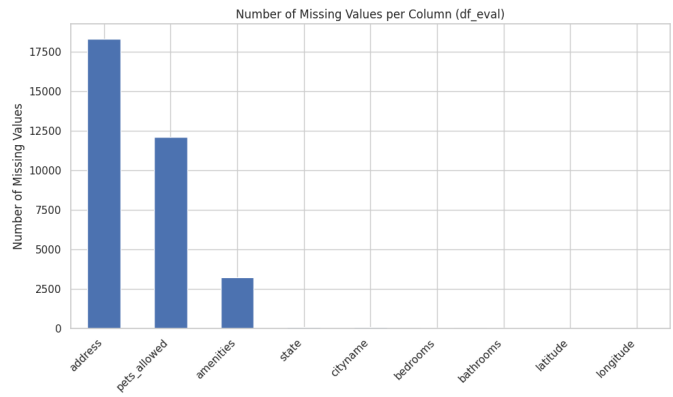

Fig. 1. Number of Missing Values per Column.

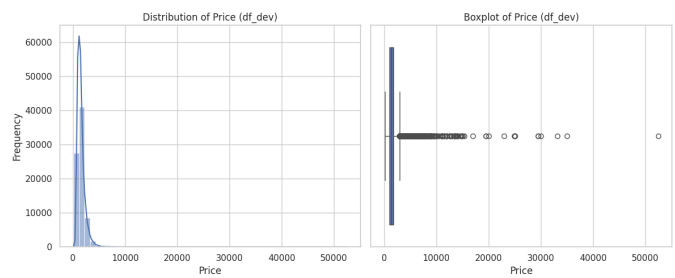

Fig. 2. Distribution of Price in the Development Set.

II. Proposed Approach

The proposed methodology encompasses a multi-stage data preprocessing pipeline, robust feature engineering, model selection, and hyperparameter optimization.

A. Preprocessing

- **Column Removal**: address (due to high missingness), currency, price_type (single unique values), and the uninformative category column were dropped. The id column was retained for submission purposes only.
- **Time Feature Engineering**: The time (Unix timestamp) was transformed into year, month, day, dayofweek, and hour to capture temporal patterns.
- **Missing Value Imputation**:
  - Numerical (bathrooms, bedrooms, latitude, longitude): Imputed with the **median** from the development set.

- Categorical location columns (cityname, state): Imputed with the **mode** from the development set.
- pets_allowed (approx. 60% missing): NaNs were filled with 'Not Specified', treating missing information as a distinct category.
- amenities (approx. 16% missing, textual): NaNs were filled with an empty string ('"), enabling consistent TF–IDF processing.

- **Boolean Feature Conversion**: fee ('Yes':1, 'No':0) and has_photo ('Yes'/'Thumbnail':1, 'No':0) were converted to integer type.
- **Outlier Handling (square_feet)**: Values were clipped at the 1st (approx. 395 sq ft) and 99th (approx. 2247 sq ft) percentiles derived from the development set. These specific percentiles were selected to effectively minimize the influence of extreme outliers without significantly discarding relevant data, thus enhancing the model's robustness and predictive stability.
- **Target Variable Transformation**: The right-skewed price target was transformed into log (numpy.log (price+1)). This normalized its distribution, mitigating the impact of extreme values and improving model performance. Predictions were converted back to original scale using numpy.expm1() for MAE calculation.

B. Feature Engineering

Beyond basic cleaning, advanced feature engineering was crucial for extracting predictive signals:

- **One-Hot Encoding**: Low-cardinality categorical features (pets_allowed, state, source) were converted into binary (0/1) columns using OneHotEncoder (handle_unknown='ignore').
- **TF–IDF Vectorization for Text**: TF–IDF Vectorizer was used to convert text data into numerical features, capturing term importance.
  - amenities: Tokenized by comma, min_df=10, max_features=200, ngram_range=(1,2).(200 features).
  - title: min_df=5, max_features=300, ngram_range=(1, 3), stop_words='english'. (300 features).
  - body: min_df=10, max_features=500, ngram_range=(1, 2), stop_words='english'. (500 features). The body column, rich in detailed descriptions, provided a substantial performance boost due to its extensive textual information. Parameters like min_df, max_features, and ngram_range were carefully chosen to balance information capture with dimensionality.
- **Target Encoding for High Cardinality Feature**: Given its high cardinality (over 2800 unique values), cityname was target-encoded using K-Fold cross-validation (K=5, shuffled, random_state=42) to prevent data leakage. This method effectively captured the average target value associated with each city, providing a powerful location-based feature. Unseen cities were imputed with the global mean of the log-transformed target.

All processed features were combined into sparse matrices using scipy.sparse.hstack, resulting in a final feature set (X_dev_sparse_body_sqft, X_eval_sparse_body_sqft) of 791 columns.

C. Model Selection and Hyperparameter Tuning

- **Performance Progression**:
  - Baseline (LGBM, default, no log transform, minimal features): MAE = **$245.37**.
  - + log target transform: MAE = **$233.63**.
  - + title TF–IDF: MAE = **$234.50** (Initially, a slight increase in MAE occurred; however, the title features were retained because subsequent tuning and interactions with other features (e.g., `body`, `cityname`) revealed their additive value in improving the final model's generalization).
  - + cityname Target Encoding: MAE = **$221.62**.
  - + body TF–IDF: MAE = **$202.45**.
  - + square_feet Outlier Handling: MAE = **$202.19**.
- **Hyperparameter Refinement**: An analytical approach combining manual adjustments and early stopping strategies proved effective for LightGBM.
  - Initial Manual Tuning: Parameters including n_estimators=500, learning_rate=0.05, subsample=0.8, colsample_bytree=0.8 (with other defaults like num_leaves=31, max_depth=-1) were applied. This configuration significantly improved MAE to **$191.38** from the default parameter baseline of **$202.19** (MAE of model with all features, but default LightGBM params).
  - Refinement with Early Stopping and Increased Estimators: To further optimize boosting rounds and prevent overfitting, n_estimators was increased to 6500, with early stopping (stopping_rounds=50) on a 20% validation set monitoring 'mae' on the log-transformed target. The model converged at **6500 iterations**, yielding a validation MAE of **$162.06**. This robust analysis demonstrated the optimal balance between model complexity and generalization.

III. Results

The final LightGBM model, configured with the optimized hyperparameters, was trained on the entire log-transformed development dataset. Predictions were then generated for the preprocessed evaluation set.

A. Prediction and Submission

The log-scaled predictions were converted back to original scale using numpy.expm1() and clipped at 0 to ensure non-negative values. Predictions were rounded to two decimal places, and a submission.csv file was generated with 'Id' and 'Predicted' columns. A sample of the submission file is provided in Table.I

TABLE I
SAMPLE OF THE FIRST FIVE ROWS FROM THE SUBMISSION FILE

Id	Predicted
0	774.75
1	1973.64
2	1107.28
3	1324.37
4	1678.53

B. Feature Importance

Feature importance analysis from the final LightGBM model (Table II) highlighted the key drivers of price prediction. cityname_encoded and square_feet were consistently the most influential, underscoring the critical roles of location and size. Geographical coordinates (longitude, latitude) and core property attributes (bathrooms, bedrooms) also ranked high. Various TF–IDF terms from body and amenities (e.g., ‘laundry‘, ‘views‘, ‘garage‘, ‘studio beds‘, ‘luxurious‘) demonstrated significant predictive power, validating the textual feature engineering.

TABLE II
TOP 20 MOST IMPORTANT FEATURES

Feature	Importance
cityname_encoded	303
square_feet	274
longitude	228
latitude	184
bathrooms	62
hour	40
bedrooms	39
state_TX	29
laundry	28
views	27
unit	24
garage	21
building	20
state_CA	20
state_OH	20
playground	20
luxurious	20
beds	19
unit number	19
urban	19

C. Model Evaluation Visualizations

Here, visualizations of the model’s performance on the validation set are presented. These plots offer a qualitative assessment of predictive accuracy and error distribution.

IV. DISCUSSION AND CONCLUSION

A. Summary of Results

This project successfully developed a LightGBM regressor for apartment rent price prediction, achieving a commendable MAE of **$162.06** on the hold-out validation set. This was accomplished through a systematic pipeline:

- **Data Cleaning & Imputation**: Addressing missing values via median/mode imputation and dedicated ’Not

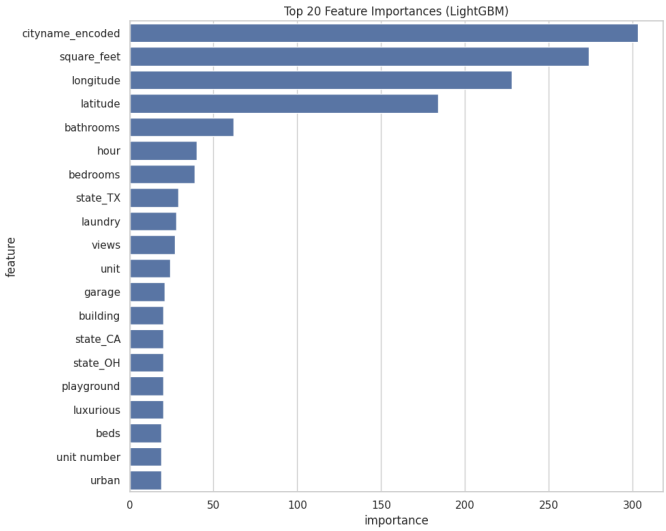

Fig. 3. Top 20 Feature Importances (LightGBM) from the final model.

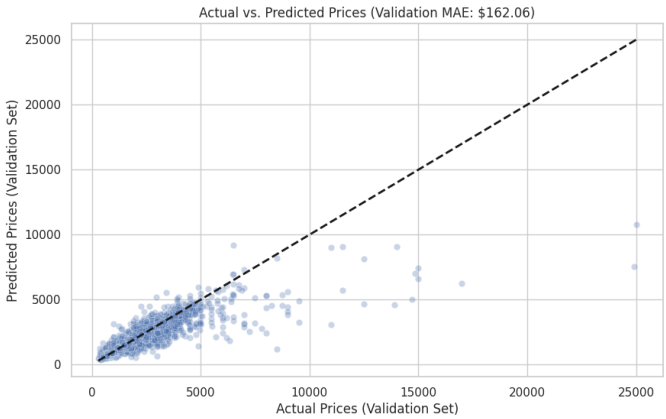

Fig. 4. Actual vs. Predicted Prices on Validation Set, showing model fit against a perfect prediction line (dashed black).

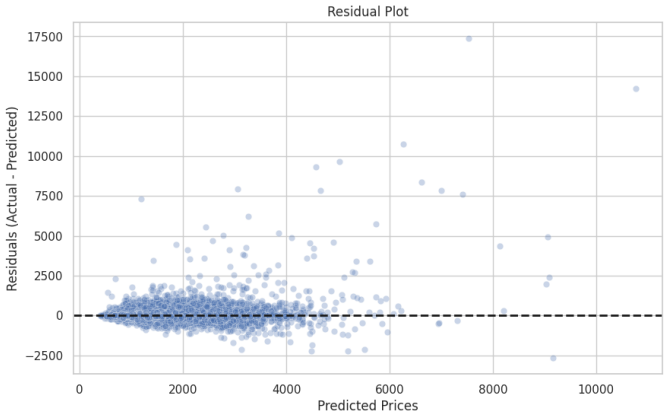

Fig. 5. Residual Plot for Validation Set Predictions. Ideally, residuals should be randomly scattered around zero, indicating no systematic errors.

Specified'/'Empty String' categories for high-missingness columns.

- **Feature Engineering**: Extraction of temporal features, comprehensive TF–IDF for text descriptions (amenities, title, and especially body for its rich detail), and robust Target Encoding for the high-cardinality cityname. The success of TF–IDF on body indicates that the detailed listing descriptions contain significant predictive signals beyond just structured features. The log transformation of the target effectively handled its skewed distribution, which is a common practice for improving regression model performance on such data.
- **Outlier Management**: Clipping square_feet outliers improved model stability and generalization, preventing extreme values from disproportionately influencing the model.
- **Model Refinement**: LightGBM's effectiveness with sparse data, combined with a well-analyzed hyperparameter set determined through iterative manual tuning and early stopping, optimized predictive accuracy. The substantial number of boosting rounds found by early stopping (6500) suggests the model effectively learned complex patterns without significant overfitting, showcasing the analytical effort in hyperparameter selection.

The feature importance analysis further confirmed the intuitive influence of location, size, and specific amenities on rent prices, validating the selected features.

B. Challenges and Limitations

- **Missing Data Management**: While addressed, the high volume of missing `address` data limited the ability to extract highly specific location features like neighborhood-level information.
- **Text Data Beyond TF–IDF**: TF–IDF effectively captured keywords, but more advanced Natural Language Processing (NLP) techniques, such as pre-trained word embeddings or transformer models, could potentially extract deeper semantic meaning and context from `title` and `body`. However, these were not pursued due to increased computational complexity and time constraints.
- **Computational Resources for Tuning**: Full-scale automated hyperparameter optimization (e.g., `RandomizedSearchCV` over a very wide search space) was constrained by computational resources, highlighting practical limitations. Manual tuning with early stopping insights was a pragmatic alternative.
- **Outlier Handling Scope**: While `price` and `square_feet` outliers were explicitly handled, other numerical features were not subjected to the same rigorous outlier treatment.
- **Generalizability**: The model's performance on future, unseen data (beyond the provided evaluation set) remains the ultimate test of its generalization, as market dynamics and listing characteristics may evolve.

C. Future Work and Potential Improvements

- **Advanced Hyperparameter Optimization**: Implement more sophisticated libraries like Optuna, allowing for efficient exploration of the hyperparameter space within a defined time budget.
- **Ensemble Modeling**: Explore combining predictions from multiple diverse models (e.g., LightGBM with CatBoost or even simpler linear models) using stacking or weighted averaging to improve robustness and potentially reduce MAE. CatBoost, in particular, could offer benefits for categorical features.
- **Deeper NLP for Text Features**: Incorporate pre-trained word embeddings (Word2Vec, GloVe, FastText) or contextual embeddings (e.g., BERT-based models for text representations) for richer semantic understanding, provided computational feasibility for feature extraction.
- **Geospatial Feature Engineering**: Develop more nuanced location-based features, such as distance to nearest city centers, public transport hubs, or points of interest, or using geospatial clustering.
- **Comprehensive Outlier Treatment**: Extend outlier analysis and treatment to other numerical features to further enhance model robustness.
- **K-Fold Cross-Validation for Training**: Implement K-Fold cross-validation for the final model training and prediction generation to increase model stability and reduce variance in predictions, especially when limited by a single train-validation split.
- **Feature Selection**: Conduct systematic feature selection techniques (e.g., based on feature importance, recursive feature elimination) to potentially reduce dimensionality, speed up training, and remove less informative or noisy features.

The developed pipeline provides a solid foundation for further enhancements and offers a practical approach to a complex prediction task. The accompanying Jupyter notebook is thoroughly commented to provide clear insights into each step of the implementation.

References

[1] Charles R. Harris et al. Array programming with NumPy. *Nature*, 585(7825):357-362, 2020.

[2] F. Pedregosa et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825-2830, 2011.

[3] Guolin Ke et al. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems*, 30, 2017.

[4] Karen Spärck Jones. A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1):11-21, 1972.